

CSC3066

Deep Learning

Anna Jurek-Loughrey



What's to come this week

- ☐ Neural Network models and text data
 - ☐ Word Embeddings
 - ☐ Word2Vec (Skip-Gram, Continuous Bag of Words)
 - ☐ Overview of other models
 - ☐ Using word embeddings for text analysis

Words Embeddings



Word Embeddings

- ☐ Word Embedding is a term used for representation of words for text analysis.
- ☐ Word embeddings are typically real-number vectors.
- ☐ Every word has a unique word embedding.
- ☐ For each word, the embedding captures the “meaning” of the word such that the words that are closer in the vector space are expected to be similar in their meaning.



Word Embeddings

- ❑ We need to find a vector space so that:
 - It can be used to embed (represent) words from a dictionary such that the vectors themselves are small (e.g. only 100-300 elements, rather than 1,000,000+ elements),
 - Words that are near one another in this multi-dimensional space have similar meanings, e.g. boy and men, child and baby, apple and banana, etc.

Words Embeddings

Word2Vec



Word Embeddings: Word2Vec Model

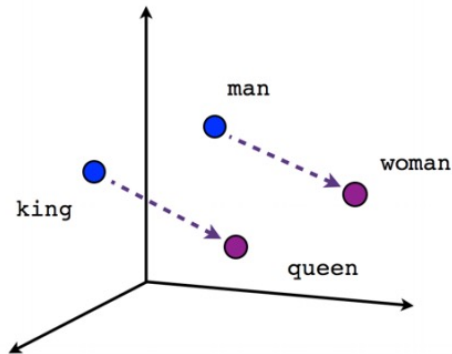
- ❑ Word2Vec Model
- ❑ Tomas Mikolov in 2013 (working at Google at the time) came up with the idea of training a simple Neural Network to learn embeddings for words based on the context in which they are used.
<https://arxiv.org/pdf/1301.3781.pdf>
- ❑ Mikolov trained a network using a dictionary of million most-frequently-used words in a training corpus consisting of the 6 billion tokens from Google News corpus.



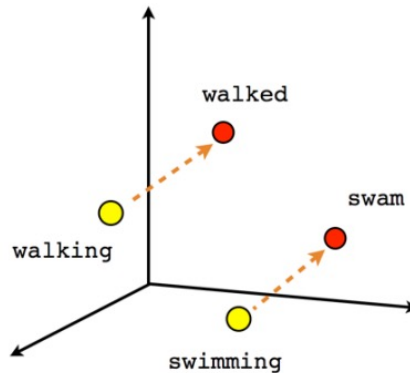
Characteristics of Word2Vec Model

- ❑ Characteristics of the Word2Vec Model
- ❑ Words with similar semantic meanings tend to group near one another in various dimensions of the space.
- ❑ Simple vector operations can be performed on the learned embedding vectors to explore these relationships.
- ❑ For example, if we take the vector representing the word “King”, subtract the vector for “Man” and add the vector for the word “Woman” we will find that the word closest to the resulting vector in the word2vec vector space turns out to be “Queen”

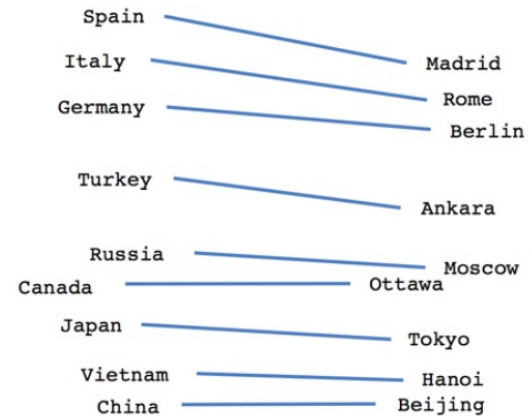
Characteristics of Word2Vec Model



Male-Female



Verb tense



Country-Capital



Word2Vec: Skip-Gram Model and CBOW Model

- ❑ There are two model architectures that can be used for generating the embedding space:
 - ❑ **Skip-Gram**: predict the surrounding (context) words from the input word.
 - ❑ **CBOW (Continuous Bag of Words)**: predicts a word based on its surrounding (context) words.

Word2Vec

Skip-gram



Word2Vec: Skip-Gram

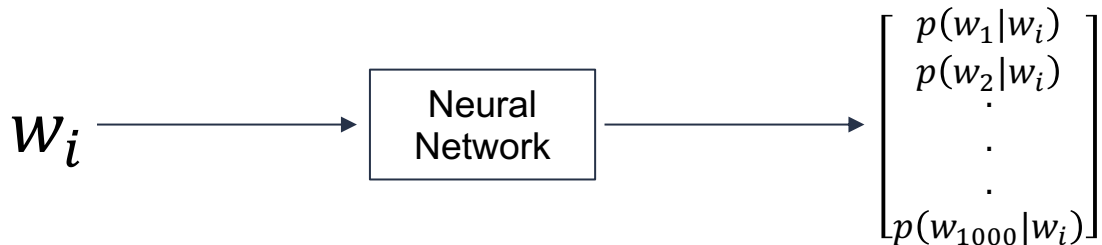
- ❑ The idea of Skip-Gram Model:
 - ❑ Given a specific word in the middle of a sentence (the input word), look at the words nearby and pick one at random.
 - ❑ We want to train an ANN so that it can calculate probability for every word in the vocabulary of being the word that was picked.
 - ❑ These probabilities relate to how likely it is to find each word from the vocabulary nearby our input word.



Word2Vec: Skip-Gram

□ The idea of Skip-Gram Model:

- For a vocabulary with 1000 words: $\{w_1, \dots, w_{1000}\}$ train an ANN model such that:





Word2Vec: Skip-Gram Model

- The idea of Skip-Gram Model:
 - For example, if we provide the network with the input word “United”, the output probabilities should be much higher for words like “Kingdom” and “State” than for unrelated words like “Apple” or “Dog”.



Word2Vec: Skip-Gram Model

□ The idea of Skip-Gram Model:

- The “nearby” (context) words are determined by a window size. For a window size 5 we are looking at 5 words behind and 5 words ahead (10 in total). This is a parameter of the model.

After the deduction of the costs of investing, beating the stock market is a loser's game.

Window of size 5Window of size 5



Word2Vec: Skip-Gram Model Training

□ The idea of Skip-Gram Model:

- The training set is constructed of pairs of words extracted from our training documents. Example for window size 2:

Source Text:

The quick brown fox jumps over the lazy dog

The quick brown fox jumps over the lazy dog

The quick brown fox jumps over the lazy dog

...

Training Samples:

(the, quick)
(the, brown)

(quick, the)
(quick, brown)
(quick, fox)

(brown, the)
(brown, quick)
(brown, fox)
(brown, jumps)



Word2Vec: Skip-Gram Model Training

- ❑ The idea of Skip-Gram Model:
 - ❑ We want the network to learn the statistics from the number of times each pair appears in the training set.
 - ❑ For example, the pair (“United”, “Kingdom”) will appear more often in the training dataset than (“United”, “Apple”).
 - ❑ Consequently, after the training, for input word “United” the network should output much higher probability for “Kingdom” than for “Apple”



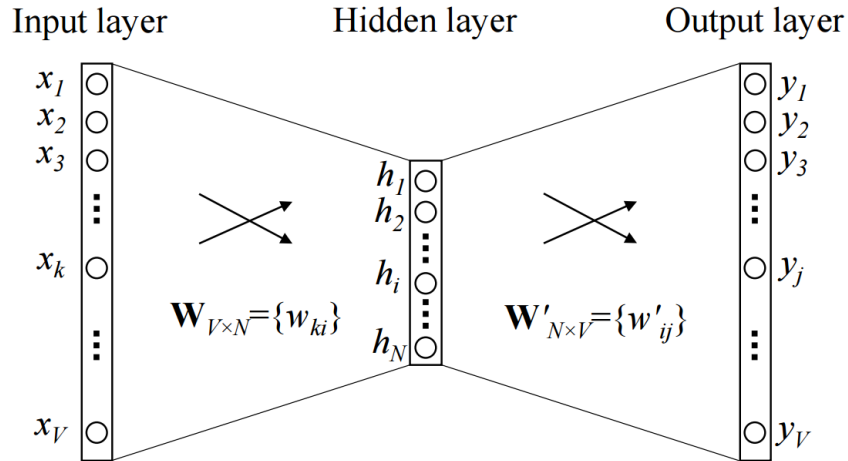
Word2Vec: Skip-Gram Model Architecture

- The idea of Skip-Gram Model:
 - Each word in the vocabulary is represented as its one-hot encoded vector.
 - For a vocabulary of size V each one-hot encoded vector will have V components (one for every word) including value 1 in the position corresponding to a specific word and values 0 in all of the other positions.
 - Consequently, each training sample is composed of two one-hot encoded vectors.



Word2Vec: Skip-Gram Model Architecture

Architecture of the Skip-Gram Model:

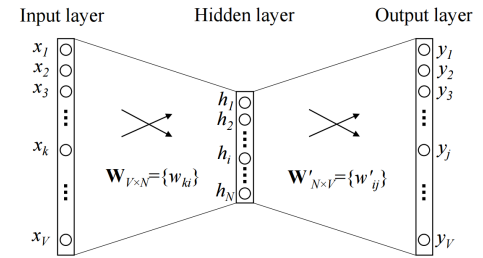




Word2Vec: Skip-Gram Model Architecture

Architecture of the Skip-Gram Model:

- There is no activation function on the hidden layer neurons.
- The output neurons use softmax activation function.
- The input to the network is a one-hot encoded vector representing the input word.
- The output is the probability distribution vector of the same dimension as the input.





Word2Vec: Skip-Gram Model Architecture

□ Skip-Gram Model - The Input Layer:

- The input of the network is a one-hot encoded vector representing the input word of dimension equal to the size of the vocabulary V .

abandon	0
ability	0
	⋮
....	0
quick	1
	0
...	⋮
zone	0



Word2Vec: Skip-Gram Model Architecture

❑ Skip-Gram Model - The Hidden Layer:

- ❑ The number of neurons in the hidden layer N determines the dimension of embedding vectors.
- ❑ The most commonly used are: 50, 100, 200 and 300
- ❑ The weights assigned to the connections between the input layer and the hidden layer are represented by a matrix of $V \times N$

$$W = \begin{bmatrix} w_{1,1} & \dots & w_{1,N} \\ \vdots & & \vdots \\ w_{V,1} & \dots & w_{V,N} \end{bmatrix}$$



Word2Vec: Skip-Gram Model Architecture

□ Skip-Gram Model - The Hidden Layer:

- If we multiply the input one-hot encoded vector $1 \times V$ by a $V \times N$ matrix, we will obtain the matrix row corresponding to value 1 in the input vector.

$$[0 \ 0 \ \dots \ 0 \ 1 \ 0 \ \dots \ 0] \times \begin{bmatrix} 2 & 0.5 & \dots & -1 \\ 0.1 & 0.2 & \dots & 0.7 \\ & \vdots & & \\ 0.4 & -0.6 & \dots & 1.3 \\ & \vdots & & \\ -1.1 & 1.8 & \dots & 2.5 \end{bmatrix} = ?$$



Word2Vec: Skip-Gram Model Architecture

□ Skip-Gram Model - The Hidden Layer:

- If we multiply the input one-hot encoded vector $1 \times V$ by a $V \times N$ matrix, we will obtain the matrix row corresponding to value 1 in the input vector.

$$[0 \ 0 \ \dots \ 0 \ \mathbf{1} \ 0 \ \dots \ 0] \times \begin{bmatrix} 2 & 0.5 & \dots & -1 \\ 0.1 & 0.2 & \dots & 0.7 \\ \vdots & \vdots & & \vdots \\ \mathbf{0.4} & \mathbf{-0.6} & \dots & \mathbf{1.3} \\ \vdots & \vdots & & \vdots \\ -1.1 & 1.8 & \dots & 2.5 \end{bmatrix} = [0.4 \quad -0.6 \quad \dots \quad 1.3]$$



Word2Vec: Skip-Gram Model Architecture

- ❑ Skip-Gram Model - The Hidden Layer:
 - ❑ The output of the hidden layer is a vector of dimension equals to the number of neurons in the hidden layer.
 - ❑ The vector is multiplied by the weights assigned to the connections between the hidden and the output layer. The weights are represented by a $N \times V$ matrix.

$$W' = \begin{bmatrix} w_{1,1} & \dots & w_{1,V} \\ \vdots & & \vdots \\ w_{N,1} & \dots & w_{N,V} \end{bmatrix}$$



Word2Vec: Skip-Gram Model Architecture

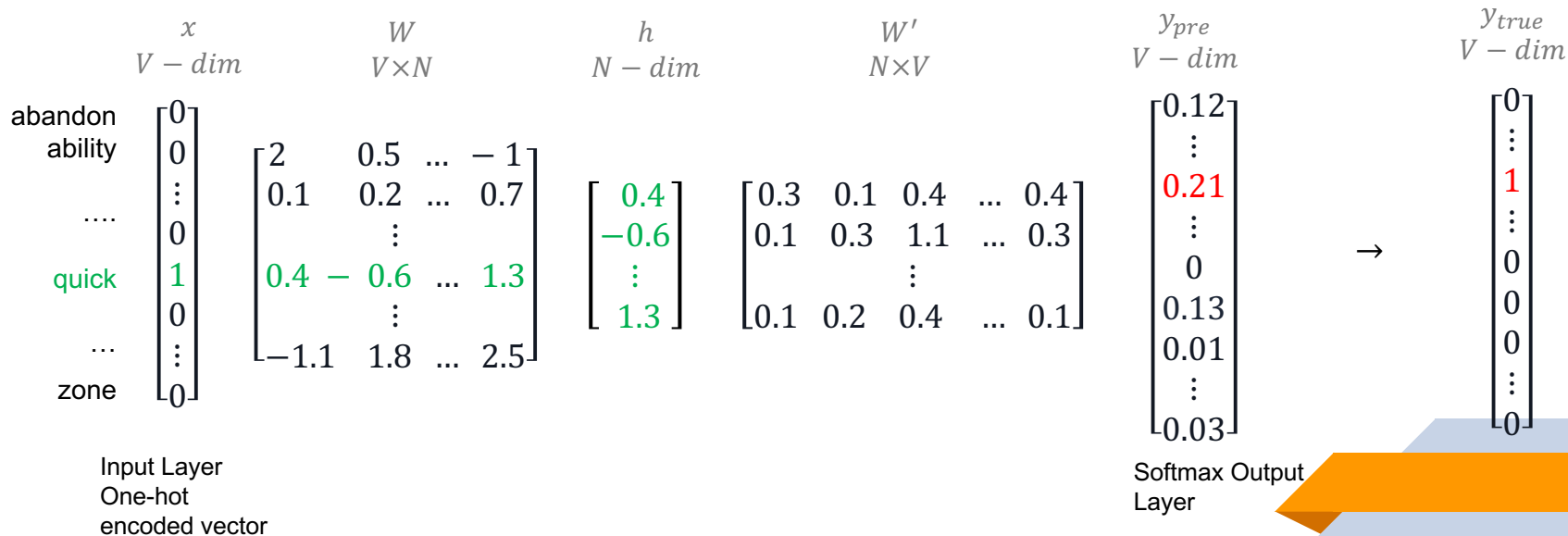
- ❑ Skip-Gram Model - The Output Layer:
 - ❑ The output layer contains V number of neurons with softmax activation function.
 - ❑ The output of the ANN model is a vector of dimension V representing probability distribution of words from the vocabulary given the input words.



Word2Vec: Skip-Gram Model Architecture

Skip-Gram Model - Architecture:

Example for (quick, fox) input





Word2Vec: Skip-Gram Model Architecture

□ Skip-Gram Model:

- The output of the hidden layer is considered as the “word vector” (embedding) for the input word.

Hidden Layer Weight Matrix

N neurons

V words



Words Vectors Table

N neurons

V words



Word2Vec: Skip-Gram Model Architecture

- ❑ Skip-Gram Model - Training:
 - ❑ The backpropagation for training samples corresponding to an input word is done in one back pass.
 - ❑ For input word “quick” ((quick, the), (quick, brown), (quick, fox)), the forward pass will be performed for each of the 3 context words.
 - ❑ The parameters will be updated based on the cumulative error.



Word2Vec: Skip-Gram Model Architecture

- ❑ Skip-Gram Model - Training:
 - ❑ Cross-Entropy is a loss function used to measure the distance between two probabilities distributions.
 - ❑ The training objective of the Skip-Gram model is to find word representations that are useful for predicting the surrounding words in a sentence or in a document.

Word2Vec

Continuous Bag of Words (CBOW)



Word2Vec: CBOW

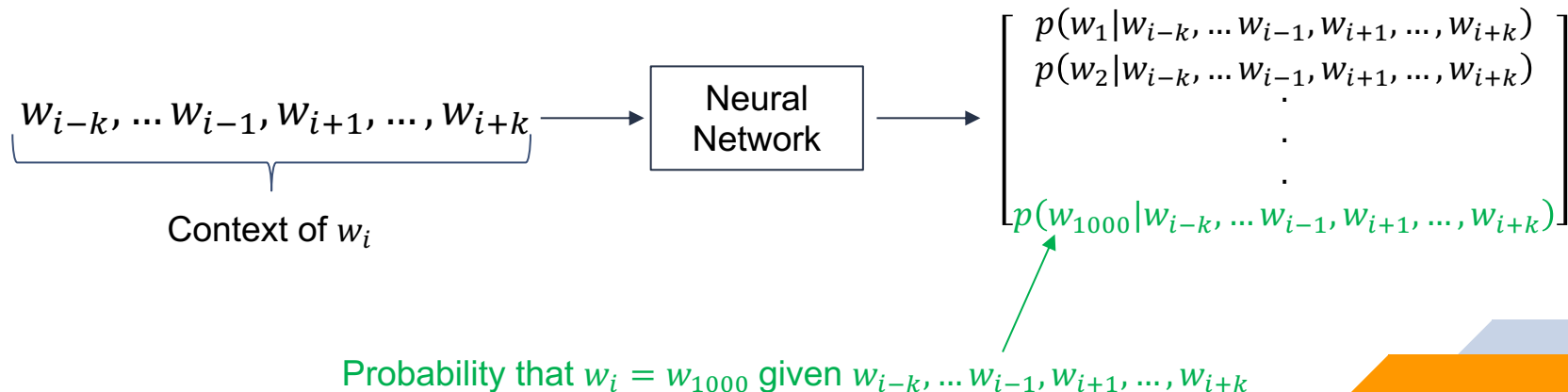
- ❑ The idea of CBOW Model:
 - ❑ Assume that we know some context (surrounding) words of an unknown target word.
 - ❑ We want to train the NN so that it can calculate probability for every word in the vocabulary of being the target word.
 - ❑ These probabilities relate to how likely each word from the vocabulary is to be the target word.



Word2Vec: CBOW

□ The idea of Skip-Gram Model:

- For a vocabulary with 1000 words: $\{w_1, \dots, w_{1000}\}$ train a Neural Network model such that:





Word2Vec: CBOW Model Training

- The idea of CBOW Model:
- The training set is constructed of pairs of the target word and its surrounding words. The number of surrounding words depend on the window size. Example for window size 2:

Source Text

The quick brown fox jumps over the lazy dog

Training Samples

([quick, brown], the)
([the, brown, fox], quick)
([the, quick, fox, jumps], brown)
([quick, brown, jumps, over], fox)
([brown, fox, over, the], jumps)
([fox, jumps, the, lazy], over)
([jumps, over, lazy, dog], the)
([over, the, dog], lazy)
([the, lazy], dog)



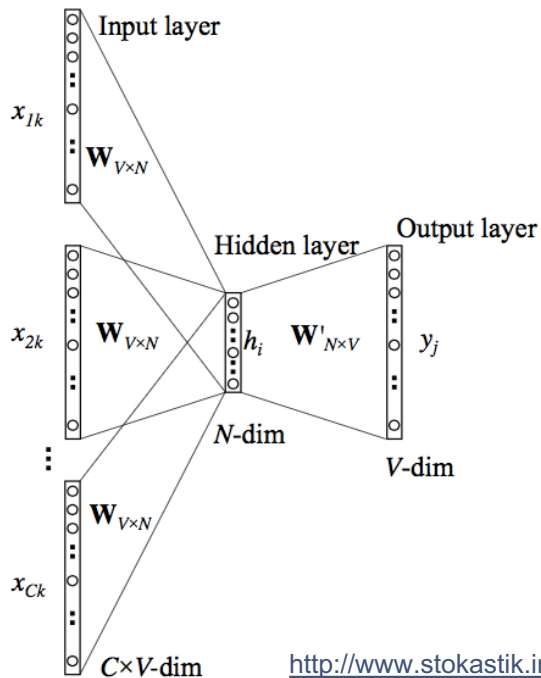
Word2Vec: CBOW Model Architecture

- The idea of CBOW Architecture:
 - Each word in the vocabulary is represented as one-hot encoded vector.
 - Consequently, each training sample is composed of a list of one-hot encoded vectors of the context words, and a single one-hot encoded vector of the target word.



Word2Vec: CBOW Model Architecture

□ The idea of CBOW Architecture:





Word2Vec: CBOW Model Architecture

- The idea of CBOW Architecture:
 - The dimensions of the hidden layer and the output layer are the same as in case of the Skip-Gram model
 - In the input layer we pass c (depends on the context window size) number, instead of one, of one-hot encoded vectors.
 - In the forward pass, each of the one-hot encoded vectors is multiplied by the weight matrix W .
 - Each multiplication gives one vector of dimension N (number of neurons in the hidden layer)



Word2Vec: CBOW Model Architecture

- The idea of CBOW Architecture:
 - All vectors are averaged element-wise to obtain the final activation of the hidden layer.
 - The final activation is then multiplied by the weights matrix W' and fed into the softmax layer.
 - The output of the hidden layer is considered as the “word vector” (embedding) for the target word.



Word2Vec: CBOW Training

- ❑ CBOW- Training:
 - ❑ Cross-Entropy is a loss function used to measure the distance between two probabilities distributions.
 - ❑ The training objective of the CBOW model is to find word representations that are useful for predicting a word by its context.



Word2Vec: Skip-Gram vs CBOW

- ❑ Skip-Gram: slower to train but works well with small amount of training data. Works better for rare words.
- ❑ CBOW: Faster to train, performs better for frequent words.



Word2Vec: Implementation in Python

- ☐ You can train your own word2vec word embedding model on your own custom corpus.
- ☐ You can load pre-trained word2vec word embedding model trained with the Google News corpus.
 - ☐ This can be done with application of the **Gensim** open source Python library for natural language processing (just one possibility).
- ☐ You can use the word2vec embeddings as the starting point and then fit them further to your task (kind of transfer learning)



Word2Vec: Implementation in Python

- ☐ You can train your own word2vec word embedding model on your own custom corpus.
- ☐ You can load pre-trained word2vec word embedding model trained with the Google News corpus.
 - ☐ This can be done with application of the **Gensim** open source Python library for natural language processing (just one possibility).
- ☐ You can use the word2vec embeddings as the starting point and then fit them further to your task (kind of transfer learning)

When would
you use one
or the other?

Python Programming Example



Words_embeddings.ipynb

Words Embeddings

Other Models



Words Embeddings with GloVe

☐ Global Vectors (GloVe)

- ☐ Another well-known model that learns embeddings of words from their co-occurrence information is GloVe.
- ☐ It was developed at Stanford University (<https://nlp.stanford.edu/projects/glove/>)
- ☐ Differently from the word2vec model, which is a ANN predictive model, GloVe is a count-based model.
- ☐ There exist GloVe word vectors pre-trained on Wikipedia articles.



Words Embeddings with GloVe

☐ Global Vectors (GloVe)

- ☐ The vectors are learnt based on a word-word co-occurrence matrix, which formulates how frequently words co-occur with one another in a given corpus.
- ☐ The main intuition underlying the model is that ratios of word-word co-occurrence probabilities have the potential for encoding some form of meaning.

Python Programming Example



Words_embeddings.ipynb



Words Embeddings with FastText

☐ FastText

- ☐ FastText is a library for learning word embeddings and text classification created by Facebook's AI Research (FAIR) lab.
- ☐ FastText embedding method is an extension of the word2vec model.
- ☐ Instead of learning vectors for words directly, FastText represents each word as an n-gram of characters.



Words Embeddings with FastText

□ FastText

- So, for example, take the word, “artificial” with $n = 3$, the FastText representation of this word is:

< art, rti, tif, ifi, fic, ici, ial >

- FastText works well with rare words. So even if a word wasn't seen during training, it can be broken down into n -grams to get its embeddings.



Embeddings with Bidirectional Encoder Representations from Transformers (BERT)

☐ BERT

- ☐ Transformer-based machine learning model developed by Google for pre-training language representations.
- ☐ It provides the basis for machine learning practitioners to build their own fine-tuned BERT-like versions to meet a wealth of different tasks.
- ☐ Pre-trained on the whole of the English Wikipedia and Books Corpus and it can be fine-tuned for different natural language processing tasks.



Using Embeddings with Machine Learning

- ❑ How can we compute a sentence/document embedding?
 - ❑ The easiest approach is to calculate embedding for each word and then accumulate them (e.g. by averaging or summing). It is a simple method but works well.
 - ❑ Instead of averaging you can also concatenate the individual word embeddings



Using Embeddings with Machine Learning

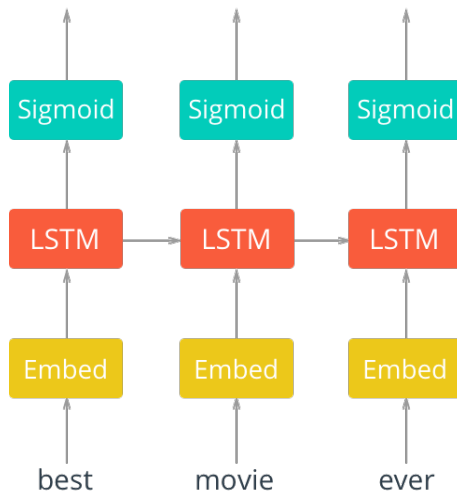
- ❑ How can we compute a sentence/document embedding?
 - ❑ You can model a document as a matrix of embeddings (Convolutional Neural Networks)

I					
like					
this					
movie					
very					
much					
!					



Using Embeddings with Machine Learning

- How can we compute a sentence/document embedding?
- You can model a document as a sequence of embeddings (Recurrent Neural Network)





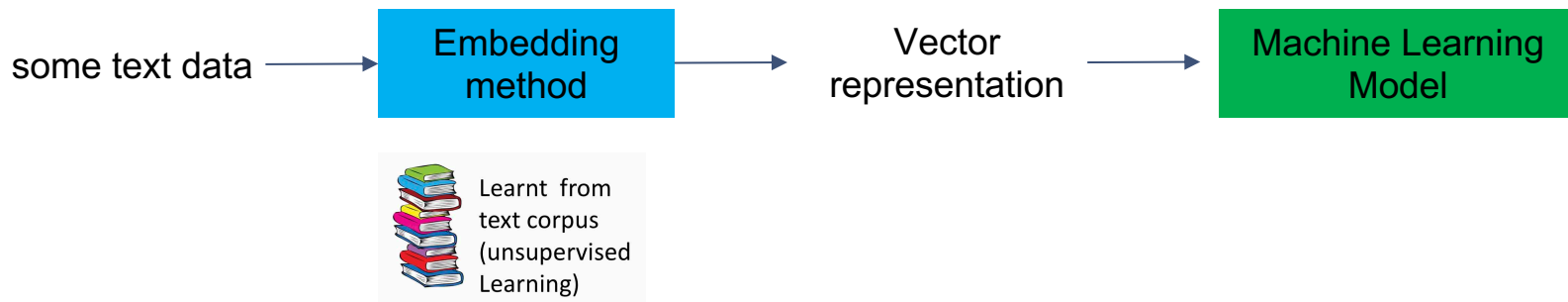
Using Embeddings with Machine Learning

- ❑ How can we compute a sentence/document embedding?
 - ❑ Some frameworks, like BERT, provide sentence and document embeddings.
 - ❑ **Context-free models** such as word2vec or GloVe generate a single word embedding representation for each word in the vocabulary. For example, the word “bank” would have the same representation in “bank deposit” and in “riverbank”. **Contextual models** instead generate a representation of each word that is based on the other words in the sentence. BERT, as a contextual model, captures these relationships in a bidirectional way.



Using Embeddings with Machine Learning

□ How do we use embeddings with ML models?





Using Embeddings with Machine Learning

- ❑ How do we use embeddings with ML models?
 - ❑ Example: sentiment analysis.
 - ❑ Collect corpus of positive and negative reviews, label them accordingly.
 - ❑ Convert each review into vector using word embeddings.
 - ❑ Train a ML model to predict whether a review is positive or negative.

Coming Next

Convolutional Neural Networks